北京郵電大學
Beijing University of Posts and Telecommunications

Queen Mary
University of London

Undergraduate Project Report
2025/26

Android Chinese Words Game App

Name:	Liu Weiliang
School:	International School
Class:	2022215107
QMUL Student No.:	221166552
BUPT Student No.:	2022213200
Programme:	**Telecommunications Engineering with Management**

Date: 06-04-2026

Table of Contents

Abstract *4*

Keywords *5*

Chapter 1: Introduction *7*

1.1 Background and Motivation **7**

1.2 Problem Statement **8**

1.3 Aim and Objectives **8**

1.4 Scope and Contributions **9**

1.5 Report Structure **9**

Chapter 2 Literature Review *11*

2.1 Gamification in Language Learning **11**

2.2 Existing Chinese Learning Applications **12**

2.3 Speech Recognition and Pronunciation Assessment Technologies **13**

2.4 Natural Language Processing in Chinese Learning Applications **13**

2.5 Mobile Educational Application Development Approaches **14**

2.6 Summary **15**

Chapter 3 Requirements Analysis *16*

3.1 Project Overview **16**

3.2 Target Users and Learning Scenarios **16**

3.3 Requirement Elicitation **16**

3.4 Functional Requirements **17**

3.4.1 User Management Requirements 17

3.4.2 Game Selection and Difficulty Requirements 17

3.4.3 Character Matching Requirements 17

3.4.4 Pronunciation Quiz Requirements 18

3.4.5 Word Puzzle Requirements 18

3.4.6 Achievement and Reward Requirements 18

3.4.7 Progress Tracking Requirements 19

3.5 Non-functional Requirements **19**

3.6 Ethical Considerations **20**

3.7 Summary **20**

Chapter 4 Design and Implementation *22*

4.1 Chapter Overview **22**

4.2 Overall System Architecture **22**

4.2.1 Layered Architecture 22

4.2.2 Activity-based Navigation 24

4.3 Core Implementation Framework **24**

4.4 Data Model and Persistence **25**

4.4.1 Database Design Rationale 25

4.4.2 DAO-based Data Access 27
4.4.3 JSON-to-SQLite Data Import 28
4.5 User Management and Game Entry 29
4.5.1 Login and Registration 29
4.5.2 Difficulty Selection and Game Start 29
4.6 Character Matching Module 30
4.6.1 Design 30
4.6.2 Implementation 31
4.7 Pronunciation Quiz Module 31
4.7.1 Design 31
4.7.2 Implementation of the Speech Workflow 32
4.8 Word Puzzle Module 33
4.8.1 Design 33
4.8.2 Implementation 33
4.9Achievement and Progress Tracking 34
4.9.1 Design 34
4.9.2 Implementation 34
4.10 UI and User Experience Considerations 36
4.11 Chapter Summary 36
Chapter 5 Results and Discussion 37
5.1 User Management Results 37
5.2 Character Matching Results 38
5.3 Pronunciation Quiz Results 39
5.4 Word Puzzle Results 40
5.5 Achievement and Progress Tracking Results 41
5.6 Discussion of System Effectiveness 42
5.7 Discussion of Limitations 42
5.8 Comparison with Initial Objectives 42
5.9 Summary 43
Chapter 6 Conclusion and Future Work 44
6.1 Overall Project Summary 44
6.2 Achievement of Objectives 44
6.3 Main Limitations 44
6.4 Future Work 45
References 46
Acknowledgement 47
Appendices 48
Disclaimer 48
Acknowledgement of using Generative AI 49
Project specification 50

Early-term progress report **51**
Mid-term progress report **52**
Supervision log **53**
Additional appendices (as needed) **54**
Risk and Environmental Impact Assessment ***55***

Abstract

This dissertation presents **CatLingo**, a gamified Android application designed to support beginner and intermediate learners of Chinese. The project responds to a practical gap in existing Chinese learning tools: although many applications provide vocabulary display, pinyin annotation, or simple drill-based exercises, fewer systems integrate character recognition, pronunciation practice, sentence-level learning, and long-term progress tracking within one coherent mobile platform. To address this problem, CatLingo combines three core mini-games—Character Matching, Pronunciation Quiz, and Word Puzzle—with account management, difficulty selection, local persistence, and achievement-based motivation. The current implementation organises these three game modes within a shared Android activity structure and uses local data storage to support persistent learner records.

From a technical perspective, the project evolved from an earlier Unity-oriented concept into a native Android solution implemented with Java and XML. The system adopts a modular structure that separates interface logic, gameplay control, and data persistence. SQLite is used to store users, scores, question details, achievements, and sentence resources, while a JSON-to-SQLite import pipeline supports content preparation and maintenance. In addition, the pronunciation module integrates speech evaluation functionality, and the Word Puzzle module relies on sentence-level token resources to generate reconstruction tasks. These design choices make the system both maintainable and suitable for iterative educational development.

The results of the project show that CatLingo has progressed beyond a conceptual prototype and now supports a complete learner workflow: users can log in, select a game and difficulty level, complete interactive language tasks, and review progress through profile and achievement pages. Character Matching provides efficient reinforcement of pinyin-to-word association, Pronunciation Quiz introduces spoken interaction and immediate evaluation, and Word Puzzle supports sentence-level understanding through structured rearrangement tasks. Together, these modules demonstrate the feasibility of integrating multiple Chinese learning activities into one Android application.

Overall, CatLingo makes a practical contribution to mobile-assisted Chinese learning by combining gamification, local data persistence, and multi-module interaction in a unified system. Although the current version remains a prototype and still requires broader user evaluation, interface refinement, and further optimisation of the pronunciation workflow, it provides a solid foundation for future development as an engaging and extensible Chinese

learning application.

Keywords

CatLingo, Android application, Chinese learning, gamification, pronunciation practice, SQLite

摘要

本文介绍了 CatLingo 的设计与实现。CatLingo 是一款面向中文初学者和中级学习者的游戏化 Android 学习应用。当前许多中文学习软件虽然提供了词汇展示、拼音标注或简单练习功能，但较少能够在同一平台中同时整合汉字识别、发音练习、句子层面的学习任务以及持续性的学习进度跟踪。为解决这一问题，CatLingo 将 Character Matching、Pronunciation Quiz 和 Word Puzzle 三个核心小游戏与用户管理、难度选择、本地数据持久化和成就激励机制结合起来，形成了一个较为完整的移动学习系统。当前实现中，这三种游戏模式已经在共享的 Android 活动结构中完成组织与切换。

在技术实现方面，本项目由早期偏向 Unity 的方案逐步演化为基于 Java 与 XML 的原生 Android 实现。系统采用模块化设计，将界面逻辑、游戏控制和数据持久化相分离。SQLite 用于存储用户信息、成绩记录、题目明细、成就数据以及句子资源，同时通过 JSON 到 SQLite 的导入机制支持题库准备与后续维护。此外，发音模块集成了语音评测能力，字词谜题模块则利用句子级词项资源生成重组任务。这些设计使系统在结构上具有较好的可维护性，也更适合作为一个可持续扩展的教育应用原型。

项目结果表明，CatLingo 已经不再停留于概念验证阶段，而是具备了较完整的学习流程：用户可以登录系统、选择游戏和难度、完成交互式语言任务，并通过个人资料页和成就页查看学习进度。Character Matching 强化了拼音与汉字之间的对应关系，Pronunciation Quiz 提供了语音输入与即时反馈，Word Puzzle 则通过句子重组任务支持句子层面的理解训练。这三个模块共同证明了在同一 Android 应用中整合多种中文学习活动的可行性。

总体而言，CatLingo 在移动辅助中文学习方面具有一定的实践意义。它通过将游戏化机制、本地数据持久化和多模块交互整合到统一系统中，为中文学习应用提供了一个可扩展的实现案例。尽管当前版本仍属于原型系统，在用户研究、界面打磨以及发音评测稳定性方面仍有进一步完善空间，但它已经为后续开发奠定了良好基础。

关键词

CatLingo，Android 应用，中文学习，游戏化，发音练习，SQLite

Chapter 1: Introduction

1.1 Background and Motivation

Mobile applications have become an important support tool for language learning because they allow learners to study at flexible times, receive immediate feedback, and repeat short practice tasks conveniently. In the field of English and other foreign language learning, many mature applications have shown that digital tools can effectively combine content delivery, learner interaction, and motivational design. However, the development of Chinese learning applications remains comparatively uneven, especially for beginner and intermediate learners.

Many existing Chinese learning tools still focus on relatively isolated functions, such as character display, pinyin annotation, or simple memorisation exercises. Although these functions are useful, they often provide limited interaction, weak progression mechanisms, and insufficient support for sustained learner motivation. As a result, learners may understand individual items but struggle to maintain interest or build a structured learning habit over time.

Chinese language learning also presents specific challenges that differ from alphabetic languages. Learners need to recognise characters, distinguish tones, connect pinyin with written words, and understand word order within sentences. These tasks require repeated exposure and active practice. Traditional repetition-based methods can support memorisation, but they are often monotonous and may discourage learners at an early stage. Therefore, there is a practical need for a learning tool that combines educational effectiveness with engaging interaction.

Gamification offers a useful direction for addressing this problem. Elements such as scores, badges, progress indicators, and level-based tasks can make repeated practice more motivating and help learners see their improvement more clearly. For this reason, this project develops an Android-based Chinese learning application that integrates language practice with game-inspired design. Instead of treating gamification as a purely visual feature, the project uses it as a way to organise learning tasks around character recognition, pronunciation practice, and sentence reconstruction.

The project, entitled Android Chinese Words Game App, aims to support beginner and intermediate learners of Chinese through three mini-games: Character Matching, Pronunciation Quiz, and Word Puzzle. In addition, the system includes user management, difficulty selection, achievement tracking, and score recording. Although the original specification proposed a Unity and C# solution, development later migrated to native Android with Java and XML,

supported by SQLite, JSON-based content loading, and selected language-processing tools. This change improved maintainability and aligned the technical implementation more closely with the actual educational and interface requirements of the project.

1.2 Problem Statement

Despite the growth of educational mobile software, many Chinese learning applications still provide limited support for active, structured, and motivating practice. In particular, they often fail to integrate several core learning needs within one coherent system. Learners may find separate tools for character recognition, pronunciation, or simple exercises, but fewer applications combine these areas with clear progression, progress tracking, and repeated interactive practice.

A further issue concerns feedback quality. For vocabulary recognition, some applications rely on static display rather than active choice-based interaction. For pronunciation, useful feedback is often unavailable unless the learner works with a teacher or external platform. At the same time, sentence-level understanding is frequently under-supported, even though word order and sentence structure are essential for practical Chinese learning. These limitations reduce both learning efficiency and long-term user engagement.

The project therefore addresses the following central question: how can an Android-based Chinese learning application be designed and implemented to improve vocabulary recognition, pronunciation practice, and sentence understanding for beginner and intermediate learners through gamified interaction, while remaining technically maintainable and educationally meaningful?

This problem involves both pedagogical and technical dimensions. Pedagogically, the application needs to provide meaningful and appropriately structured tasks. Technically, the system must coordinate user accounts, local storage, question selection, achievement logic, and pronunciation-related functionality within a manageable Android architecture. The project aims to respond to both dimensions through the design of a modular, game-based learning application.

1.3 Aim and Objectives

The aim of this project is to design and implement a gamified Android application that supports beginner and intermediate learners of Chinese in practising character recognition, pronunciation, vocabulary, and sentence ordering.

To achieve this aim, the project has five main objectives. First, it identifies the needs of target learners and converts them into functional and non-functional requirements. Second, it designs a modular Android architecture that supports user management, game selection, difficulty control, and local data persistence. Third, it implements three educational mini-games, namely Character Matching, Pronunciation Quiz, and Word Puzzle, each targeting a different aspect of Chinese learning. Fourth, it integrates a database and structured content management mechanism to store users, scores, achievements, and question banks. Fifth, it introduces gamification features such as achievements, progress statistics, and score recording in order to support long-term motivation and repeated use.

These objectives are consistent with the original project specification and the progress recorded in the early-term and mid-term reports. They also provide the basis for the design, implementation, and later evaluation of the system.

1.4 Scope and Contributions

The scope of this project is limited to an Android-based educational application for beginner and intermediate Chinese learners. The system focuses on foundational learning tasks, including pinyin-character matching, pronunciation practice, and simple sentence reconstruction. It is not intended to provide a complete Chinese curriculum, advanced grammar tutoring, conversational AI, or cloud-based synchronisation. Instead, it is designed as a local, modular learning prototype that demonstrates a practical and extensible solution.

Within this scope, the project makes four main contributions. First, it develops an integrated Chinese learning application that combines multiple learning activities in one system rather than treating them as isolated functions. Second, it demonstrates how gamification can be applied to Chinese learning tasks in a targeted way. Third, it shows the design of a maintainable native Android architecture using Java, XML, SQLite, DAO-based persistence, and JSON-based content preparation. Fourth, it explores the practical integration of pronunciation evaluation and sentence preprocessing into an educational mobile application.

1.5 Report Structure

The remainder of this dissertation is organised as follows. Chapter 2 reviews the relevant literature, including gamification in language learning, Chinese learning applications, pronunciation assessment technologies, natural language processing, and mobile educational app development. Chapter 3 presents the requirements analysis, including target users,

functional requirements, non-functional requirements, and ethical issues. Chapter 4 explains the design and implementation of the system, including architecture, database design, and the design and implementation of the major functional modules. Later chapters present testing, results, discussion, conclusion, and future work.

Chapter 2 Literature Review

2.1 Gamification in Language Learning

Gamification refers to the use of game-inspired elements in non-game contexts to increase engagement and persistence. In educational applications, common gamification mechanisms include points, badges, progress bars, levels, rewards, and task-based challenges. Research in language learning suggests that these mechanisms can be effective when they are linked directly to meaningful learning activities rather than added only for entertainment value.

Language learning particularly benefits from gamification because repetition is necessary for vocabulary acquisition, pronunciation practice, and pattern recognition. Without an engaging structure, repeated exercises can feel monotonous and learners may abandon the activity quickly. Gamified systems can reduce this problem by turning repeated practice into short, goal-oriented interactions. Progress indicators and rewards can also help learners perceive improvement more clearly, which is important for long-term motivation.

However, the literature also shows that gamification is not automatically effective. If reward mechanisms are disconnected from learning goals, users may focus on collecting points instead of improving language skills. Similarly, poorly designed difficulty levels can create either frustration or boredom. Therefore, successful gamification requires balanced task design, appropriate challenge, and timely feedback.

These observations are directly relevant to the present project. In this system, game mechanics are embedded in the learning tasks themselves. Character Matching links recognition with score-based progression. Pronunciation Quiz transforms speaking practice into an interactive evaluation task. Word Puzzle uses sentence reconstruction as a challenge mechanism. The achievement system then connects repeated use with visible progress. In this sense, the application follows the idea that gamification should support learning rather than distract from it.

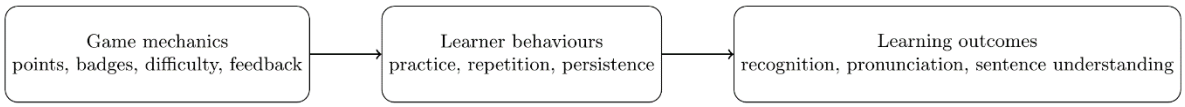


Figure 2.1 Gamification mechanism in CatLingo

2.2 Existing Chinese Learning Applications

Existing language learning applications provide useful reference points for the design of Chinese educational software. Duolingo is one of the best-known examples of gamified language learning. It uses short tasks, visible progress, points, streaks, and structured levels to encourage frequent interaction. Its success demonstrates the importance of simple tasks, low entry barriers, and immediate feedback.

Baicizhan provides another useful example, particularly for vocabulary learning. Its design emphasises staged repetition, visual support, and focused memorisation. Compared with more general language learning platforms, it shows the value of concentrating on one learning objective while keeping the interaction manageable and regular.

Nevertheless, many Chinese learning applications still have several limitations. Some provide pinyin or stroke-order display but offer little active interaction. Others include quiz functions but have weak reward systems or inconsistent difficulty progression. Pronunciation-related features may also provide limited or unclear feedback, leaving learners unsure how to improve. As a result, the learner experience may remain fragmented.

These limitations justify the need for the current project. The *Android Chinese Words Game App* attempts to combine several features that are often separated across different tools: character recognition, pronunciation practice, sentence reconstruction, score tracking, achievement display, and user-centred progress records. This integrated design aims to produce a more coherent learning experience.

Table 2.1 Comparison of representative language learning applications

Application	Main learning focus	Gamification elements	Strengths	Limitations for Chinese learning	Relevance to CatLingo
Duolingo	General language learning with short skill-based tasks	Points, streaks, hearts, levels, daily goals	Strong habit formation, clear task flow, immediate feedback	Chinese support is broad but not specialised for pinyin-character mapping or sentence reconstruction	Shows how micro-learning and visible progress can maintain engagement
Baicizhan	Vocabulary memorisation and repeated review	Progress tracking, staged review, achievement-like progression	Strong lexical focus, regular repetition, visual support	Focuses more on memorisation than integrated pronunciation and sentence reordering	Useful reference for vocabulary-centred Chinese learning
CatLingo	Character recognition, pronunciation, and sentence reconstruction	Scores, difficulty levels, progress tracking, achievements	Integrates three learning tasks in one Android app with local persistence and structured question data	Prototype stage, still needs final testing and refinement	Serves as the project's own target system

2.3 Speech Recognition and Pronunciation Assessment Technologies

Pronunciation is one of the most difficult aspects of Chinese learning, especially for non-native speakers who must control initials, finals, tones, and fluency at the same time. In conventional settings, pronunciation quality is often evaluated by teachers. In mobile learning environments, however, learners need technological support that can provide fast and accessible feedback.

Speech recognition and pronunciation assessment technologies therefore play an important role in computer-assisted language learning. Such systems typically analyse spoken input and compare it with a target item. They may return an overall score or several component measures such as accuracy, fluency, completeness, or tone-related quality. Multi-dimensional feedback is useful because it gives the learner more information than a simple correct-or-incorrect judgment.

For Chinese, pronunciation assessment is especially important because tonal errors can change meaning even when the segmental pronunciation is close to correct. This makes Chinese pronunciation evaluation more demanding than simple sound matching. In practice, educational applications often rely on third-party evaluation engines rather than attempting to build a full custom speech model.

The present project follows this practical approach. The Pronunciation Quiz module integrates speech evaluation support into the Android application and converts the returned results into learner-facing feedback and game scores. This makes pronunciation assessment part of the learning flow rather than a separate external activity.

At the same time, the literature highlights an important limitation: speech evaluation systems may be affected by accent bias, speaking conditions, and differences between learners and the model's training distribution. For this reason, pronunciation results in educational software should be presented as supportive feedback rather than absolute judgment. This principle is reflected in the current project's design.

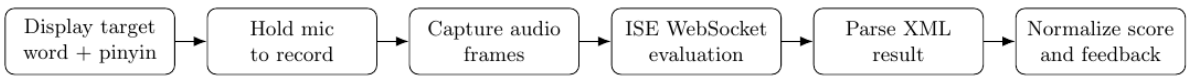


Figure 2.2 Pronunciation quiz workflow

2.4 Natural Language Processing in Chinese Learning Applications

Natural Language Processing is particularly relevant in Chinese educational applications because written Chinese does not separate words with spaces in the same way as English. For sentence-level learning tasks, segmentation is usually required before the text can be

manipulated computationally.

In this project, NLP is most directly related to the Word Puzzle module. To create a sentence reconstruction task, the system must first divide a sentence into meaningful units and then shuffle them into a puzzle. This makes sentence segmentation a necessary preprocessing step. Using NLP support for this task improves efficiency and allows sentence resources to be converted into puzzle content more systematically.

Chinese segmentation is not always straightforward. Word boundaries can depend on context, and a purely technical segmentation result may not always be pedagogically clear. In educational applications, learner clarity is especially important. Puzzle units must therefore be understandable and useful for the target proficiency level.

The current project addresses this by processing sentence content in advance and storing structured results in the local database. This reduces runtime cost on the mobile device and makes the Word Puzzle module more efficient during gameplay. It also supports later extensions such as difficulty classification based on sentence length, vocabulary frequency, or word type.

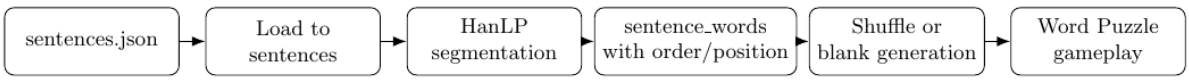


Figure 2.3 NLP support for Word Puzzle generation

2.5 Mobile Educational Application Development Approaches

Educational mobile applications can be developed using different technical approaches, including cross-platform frameworks, game engines, and native platform tools. Each approach offers advantages and limitations in terms of interface control, maintainability, performance, hardware integration, and development effort.

In the early stage of this project, Unity and C# were considered because the system included several mini-games and a gamified learning design. Unity is strong for game-oriented interaction and visual development. However, as the project evolved, a native Android approach became more suitable. The application required login forms, profile pages, database integration, activity-based navigation, and relatively lightweight educational interactions rather than complex animation-heavy gameplay.

Native Android development with Java and XML offered better alignment with these needs. It made interface design more direct, database integration more manageable, and modular structure easier to maintain. As a result, the project migrated from the original Unity-based plan

to a native Android implementation. This decision improved consistency between the project's educational goals and its technical solution.

Table 2.2 Comparison of mobile development approaches for this project

Approach	Advantages	Disadvantages	Suitability for CatLingo
Unity + C#	Good for game scenes, animation, and cross-platform game prototyping	Less natural for form-heavy UI, activity navigation, and SQLite-centred workflows	Medium
Native Android (Java + XML)	Direct Android UI control, easier activity lifecycle management, simpler local DB integration, better maintainability	Android-specific and less convenient for future cross-platform release	High

2.6 Summary

This chapter reviewed the literature most relevant to the project. It showed that gamification can improve language learning when it is connected to meaningful educational tasks. It also highlighted the limitations of many existing Chinese learning applications, especially in relation to interactivity, progression, and feedback. In addition, the chapter discussed pronunciation assessment technologies, Chinese NLP for sentence-based learning, and the practical reasons for choosing a native Android implementation. Together, these ideas provide the theoretical and technical foundation for the requirements and design decisions presented in the following chapters.

Chapter 3 Requirements Analysis

3.1 Project Overview

The purpose of this project is to build an Android-based Chinese learning application for beginner and intermediate learners. The system combines educational tasks with game-based interaction in order to improve engagement and support repeated practice. The application is designed not as a general-purpose course platform, but as a focused learning tool centred on three mini-games and a set of supporting user and data functions.

From a system perspective, the project consists of five major parts: user management, game selection and difficulty control, learning game modules, persistent data management, and achievement plus progress tracking. These parts work together to create a learning environment in which the learner can sign in, select activities, complete tasks, receive feedback, and accumulate progress over time.

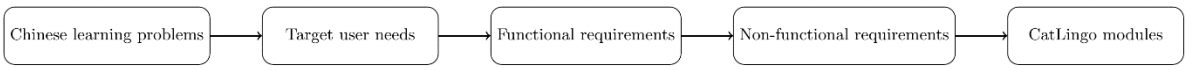


Figure 3.1 Requirement analysis framework

3.2 Target Users and Learning Scenarios

The target users are beginner and intermediate learners of Chinese. These users may already know some basic vocabulary or pinyin, but they still need support in three main areas: recognising Chinese words, producing accurate pronunciation, and understanding simple sentence structure. They are unlikely to need advanced grammar tutoring or academic-level reading support at this stage.

Three main usage scenarios were considered during requirement analysis. The first is short daily vocabulary practice, in which a learner opens the app for a brief session and completes a few recognition tasks. The second is focused pronunciation practice, in which the learner wants immediate feedback on a spoken word or sentence. The third is sentence-level review, in which the learner reconstructs simple Chinese sentences from shuffled parts. These scenarios directly motivated the design of the three mini-games and the need for short, mobile-friendly interactions.

3.3 Requirement Elicitation

The system requirements were derived from three main sources. The first source was the

educational difficulty of Chinese language learning, especially for beginners. The second source was the review of existing language learning applications and the identification of their strengths and limitations. The third source was the project's own development process, including the task specification and the refinement recorded in the early-term and mid-term reports.

This means that requirement elicitation in this project was practical and iterative rather than survey-based. Requirements were first outlined in the project specification, then clarified during implementation planning, and later refined as the technical direction changed from Unity to native Android. This process is appropriate for an undergraduate implementation project because the final system must remain grounded in both learner needs and feasible engineering choices.

3.4 Functional Requirements

The functional requirements define what the system must do in order to support its intended users and learning tasks. For clarity, these requirements are grouped into seven categories corresponding to the main user-facing and internal functions of the application.

3.4.1 User Management Requirements

The system shall allow users to register, log in, update passwords, and view profile information. Each user account shall be associated with stored learning data so that progress can be maintained across sessions. The profile shall display basic user information and summary statistics such as total score and games played.

3.4.2 Game Selection and Difficulty Requirements

The system shall provide a central game selection interface through which users can access Character Matching, Pronunciation Quiz, and Word Puzzle. Before entering a game session, users shall be able to select a difficulty level. Difficulty levels shall influence the content presented in each game mode.

3.4.3 Character Matching Requirements

The Character Matching module shall provide a question-answer activity in which the learner matches a pinyin prompt to the correct Chinese word. For each question, the system shall display a target pinyin item and a set of options from which the user selects the correct answer. The system shall support real-time answer checking and immediate correctness feedback.

A complete game session shall consist of multiple questions, with session progress clearly displayed to the user. The question set shall be generated dynamically from stored data rather than being fully hard-coded. The system shall support distractor generation so that incorrect options remain plausible and educationally meaningful.

The system shall record the user's performance in this game, including score, accuracy, and per-question details where relevant.

3.4.4 Pronunciation Quiz Requirements

The Pronunciation Quiz module shall allow the learner to pronounce a displayed word or sentence and receive feedback on pronunciation quality. The system shall show the target item clearly and provide a recording mechanism through which the user can speak into the device microphone.

The system shall transmit or process the recorded speech using an integrated pronunciation assessment component. After evaluation, it shall provide feedback to the user in a form that is understandable and useful.

The module shall also support retry interaction so that the user can repeat an attempt if necessary. Performance results shall be recorded for later progress tracking.

3.4.5 Word Puzzle Requirements

The Word Puzzle module shall provide a sentence reconstruction activity in which the learner rearranges shuffled word units into the correct order. The system shall select a sentence from the question bank, process it into puzzle elements, shuffle the elements, and present them to the learner as the input for the task.

The module shall support user interaction for rearranging the puzzle units. Depending on implementation constraints, this may use drag-and-drop interaction, tap-to-reorder interaction, or another intuitive mechanism. The system shall check whether the final arrangement matches the correct order and return feedback accordingly.

The module shall also support different difficulty levels, which may depend on sentence length, word complexity, or other linguistic properties. User performance shall be recorded for later analysis and progress display.

3.4.6 Achievement and Reward Requirements

The system shall include a reward mechanism to encourage user engagement. Users shall be

able to unlock virtual achievements or badges by meeting defined conditions. These conditions may include cumulative milestones such as number of games played, consecutive login days, or game-specific performance targets.

Achievements shall be stored persistently and linked to the corresponding user account. The system shall provide a dedicated interface or section through which the user can view unlocked achievements and, where appropriate, the progress towards future achievements.

This requirement supports the project's gamification objective by providing visible signs of progress and reinforcing repeated use.

3.4.7 Progress Tracking Requirements

The system shall track game results, including at least score, game type, difficulty level, play date, and summary performance. Where appropriate, the system shall also store more detailed per-question information, such as user answer, correctness, and time spent.

The application shall present relevant progress information to the learner through the profile or achievement-related interfaces. This information shall help users monitor their own development over time and recognise which activities they have completed most frequently.

Progress tracking is essential not only for motivation but also for educational continuity. It transforms the system from a collection of isolated mini-games into a coherent learning tool.

Table 3.1 Summary of functional requirements

Requirement ID	Description	Related module
FR-1	Users can register, log in, and change passwords	MainActivity, register, update_test
FR-2	Users can view and update profile information	UserProfileActivity
FR-3	Users can choose one of three games from the hub page	game_choice
FR-4	Users can select EASY, MEDIUM, or HARD before starting a game	game_choice
FR-5	Character Matching shows pinyin and multiple options with immediate feedback	GameActivity
FR-6	Pronunciation Quiz records speech and returns evaluation feedback	GameActivity, IseEvaluator
FR-7	Word Puzzle generates reorder or blank-style sentence tasks	GameActivity, sentence resources
FR-8	Session scores and per-question details are stored persistently	game_scores, game_question_details
FR-9	JSON data can be reloaded into SQLite during development or testing	DataManager, SeedDataLoader
FR-10	Users can view unlocked achievements	UserAchievementsActivity

3.5 Non-functional Requirements

The system shall provide a clear and easy-to-use interface suitable for early-stage learners. It shall respond to user actions without excessive delay and operate reliably under normal usage conditions. It shall also be maintainable, meaning that UI logic, game logic, and data access

should be reasonably separated. Finally, it shall remain compatible with the Android platform and usable across typical device configurations within the project scope.

3.6 Ethical Considerations

The project involves several ethical considerations even though it is not based on large-scale human subject experimentation. The most important considerations concern user data, AI-assisted evaluation, and learner fairness.

The first ethical issue is **data privacy**. The application stores user information such as usernames, passwords, emails, scores, and achievement records. Even in a prototype system, such data should be handled responsibly. Local storage reduces exposure compared with a cloud-based system, but it still requires careful management of sensitive information.

The second issue is **voice data processing**. The Pronunciation Quiz relies on third-party speech evaluation technology. Users should therefore be informed that their spoken input may be processed by an external evaluation service or SDK. Although the application is educational in intent, the transparency of processing remains important.

The third issue is **algorithmic bias and fairness**. Pronunciation evaluation systems may not always perform equally well for all learners, especially non-native speakers with different accents and speaking habits. A low score from the system does not necessarily represent absolute speaking ability. This means that evaluation feedback should be presented as learning support rather than definitive judgment.

Overall, the project addresses ethical issues through transparency, moderate use of AI-based evaluation, and the design intention to support learning rather than to rank or penalise users.

3.7 Summary

This chapter has defined the requirements that guide the development of the Android Chinese learning application. It first described the project context and target learners, then identified the main usage scenarios that motivated the system design. It explained how the requirements were elicited through educational analysis, review of existing applications, and iterative project refinement.

The functional requirements were grouped into user management, game selection and difficulty, Character Matching, Pronunciation Quiz, Word Puzzle, achievements, and progress tracking. These requirements collectively define the educational and technical expectations of the

application. The chapter also identified the key non-functional requirements, including usability, performance, reliability, maintainability, and compatibility. Finally, the ethical implications of user data, speech evaluation, and fairness were discussed.

These requirements provide the foundation for the next chapter, which explains how the system was designed and implemented to satisfy them.

Chapter 4 Design and Implementation

4.1 Chapter Overview

This chapter explains how **CatLingo** was designed and implemented to satisfy the functional and non-functional requirements defined in Chapter 3. In line with the project guidance and the style of the provided examples, this chapter focuses on system structure, module design, algorithm flow, database organisation, and representative implementation details, instead of presenting finished screenshots of the application interface. The chapter therefore combines textual explanation, flowcharts, tables, and short code snippets to show how the final Android application was built.

The discussion is organised in five parts. First, the chapter explains the overall system architecture and the decomposition of CatLingo into functional modules. Second, it discusses database design and the data-loading mechanism from JSON into SQLite. Third, it presents the design and implementation of user management and game selection. Fourth, it explains the implementation of the three core learning modules: Character Matching, Pronunciation Quiz, and Word Puzzle. Finally, it describes the achievement and progress-tracking mechanism that connects repeated gameplay with persistent learner records.

Compared with the earlier draft, this version of the chapter uses more implementation evidence directly grounded in the repository, including the activity flow in MainActivity, the difficulty dialog and loading workflow in game_choice, the central mode dispatch in GameActivity, the JSON-to-SQLite data reload process in DataManager, the HanLP-assisted sentence loading in SeedDataLoader, and the WebSocket-based pronunciation evaluation in IseEvaluator.

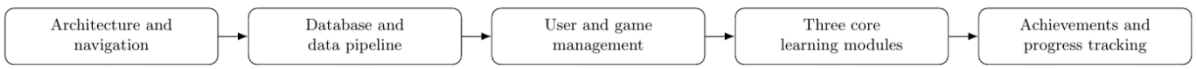


Figure 4.1 Chapter structure of design and implementation

4.2 Overall System Architecture

4.2.1 Layered Architecture

CatLingo adopts a modular native Android architecture. At a high level, the system can be understood as four layers: the presentation layer, the control layer, the data layer, and the external service layer.

The **presentation layer** contains the activities and XML layouts that interact directly with the

learner. This includes the login page, game-selection page, profile page, achievement page, and the shared game page layout. For example, activity_main.xml implements the login card and the four main actions, while activity_game_choice.xml contains the three game buttons and the toolbar menu. The shared activity_game.xml layout contains three main panels: Character Matching, Word Puzzle, and Pronunciation Quiz.

The **control layer** contains the activity logic that coordinates user interaction, game progression, and transitions between modules. MainActivity handles login, register, password update, and JSON reload. game_choice manages difficulty selection and page navigation. GameActivity is the core controller of the application and dispatches behaviour according to the received GAME_TYPE.

The **data layer** is based on SQLite, DAO classes, and local data managers. Persistent storage is used for users, achievements, game scores, per-question details, sentence resources, and word-level resources. Data access is intentionally separated from the activity layer in order to improve maintainability and reusability. This design is consistent with the DAO-based structure shown in the repository and avoids embedding SQL directly in most user-interface methods.

The **external service layer** includes the pronunciation evaluation and linguistic preprocessing components. The speech module is implemented through IseEvaluator, which connects to the iFlyTek ISE service via WebSocket and returns XML-based evaluation results. Sentence preprocessing is supported through HanLP segmentation and word-position updating in SeedDataLoader.

This layered design improves separation of concerns. Interface changes can be made without redesigning the database, and content resources can be updated through JSON files and data-loading logic without changing the core activity structure.

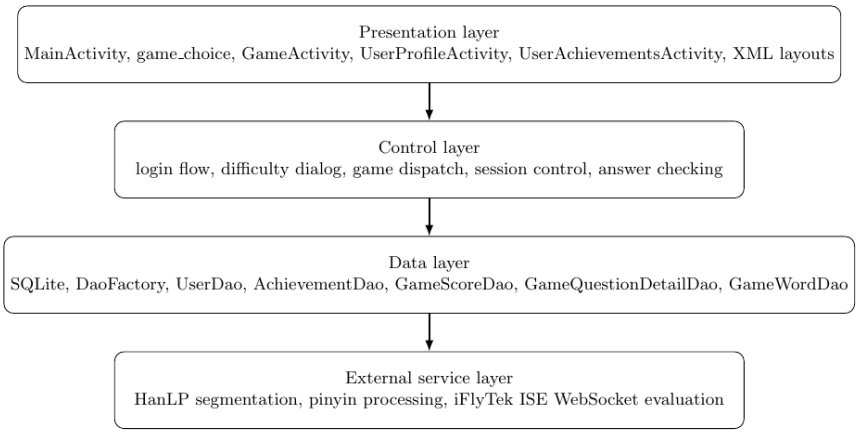


Figure 4.2 Overall system architecture of CatLingo

4.2.2 Activity-based Navigation

The navigation structure of CatLingo follows a standard Android activity flow. The learner first enters MainActivity, where the app supports login, registration, password update, and JSON reload. After successful login, the user is redirected to game_choice, where the three game modes can be selected. The menu in the same page also provides access to UserProfileActivity, UserAchievementsActivity, and logout. Once a game is selected, GameActivity is started with the corresponding GAME_TYPE, USER_ID, and DIFFICULTY.

This structure is suitable for the current project because it keeps the navigation simple and reduces unnecessary fragmentation. The learner always returns to the same hub page after completing or leaving a game, which supports repeated short practice sessions.

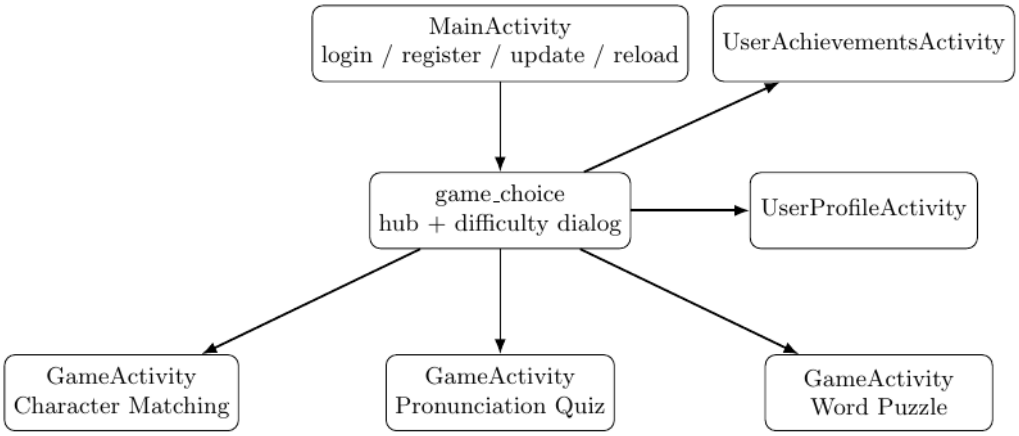


Figure 4.3 Activity navigation flow of CatLingo

4.3 Core Implementation Framework

One of the most important design decisions in CatLingo is the use of a **single container activity** to support multiple learning games. Instead of creating a completely separate activity for each game mode, the project uses GameActivity as a controller that switches interface panels and logic according to the incoming GAME_TYPE.

When the activity starts, it reads GAME_TYPE and USER_ID from the intent. It then activates one of three branches. For CHARACTER_MATCHING, it loads the option-based recognition interface and starts the matching session. For PRONUNCIATION_QUIZ, it loads the pronunciation panel, initialises the speech subsystem, and then starts the speaking session. For WORD_PUZZLE, it activates the word-puzzle panel and starts the puzzle-generation process.

```
if ("CHARACTER_MATCHING".equals(gameType)) {
panelCharacterMatching.setVisibility(View.VISIBLE);
```

```
startCharacterMatchingGame();
} else if ("PRONUNCIATION_QUIZ".equals(gameType)) {
panelPronunciationQuiz.setVisibility(View.VISIBLE);
initializeSpeechRecognition();
startPronunciationQuizGame();
} else if ("WORD_PUZZLE".equals(gameType)) {
panelWordPuzzle.setVisibility(View.VISIBLE);
startWordPuzzleDragGame();
}
```

This design makes the codebase more compact and keeps the shared session-management logic in one place. It also aligns well with the current XML structure in activity_game.xml, where all three panels are already defined in the same layout and shown or hidden dynamically.

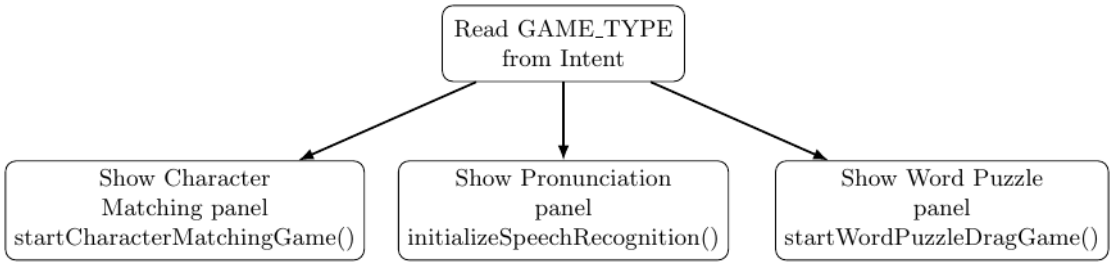


Figure 4.4 Mode dispatch inside GameActivity

4.4 Data Model and Persistence

4.4.1 Database Design Rationale

CatLingo relies on local SQLite persistence because the application needs to store user identity, question resources, game records, achievements, and progress statistics without requiring a remote server. This is appropriate for an undergraduate educational prototype, since the project focuses on local learning interaction rather than cloud synchronisation.

The data model contains several groups of tables. User-related data are stored in users, while achievement-related information is stored in achievements and user_achievements. Session-level performance is stored in game_scores, and per-question details are stored in game_question_details. Sentence resources are stored in sentences and sentence_words, while game-specific question resources are represented through character_matching, pronunciation_quiz, and word_puzzle. This structure is consistent with the database-oriented design described in your earlier report and with the import logic implemented in the codebase.

The design balances normalisation and practicality. Session-level and question-level data are separated so that the system can support both summary statistics and more detailed analytics in later chapters.

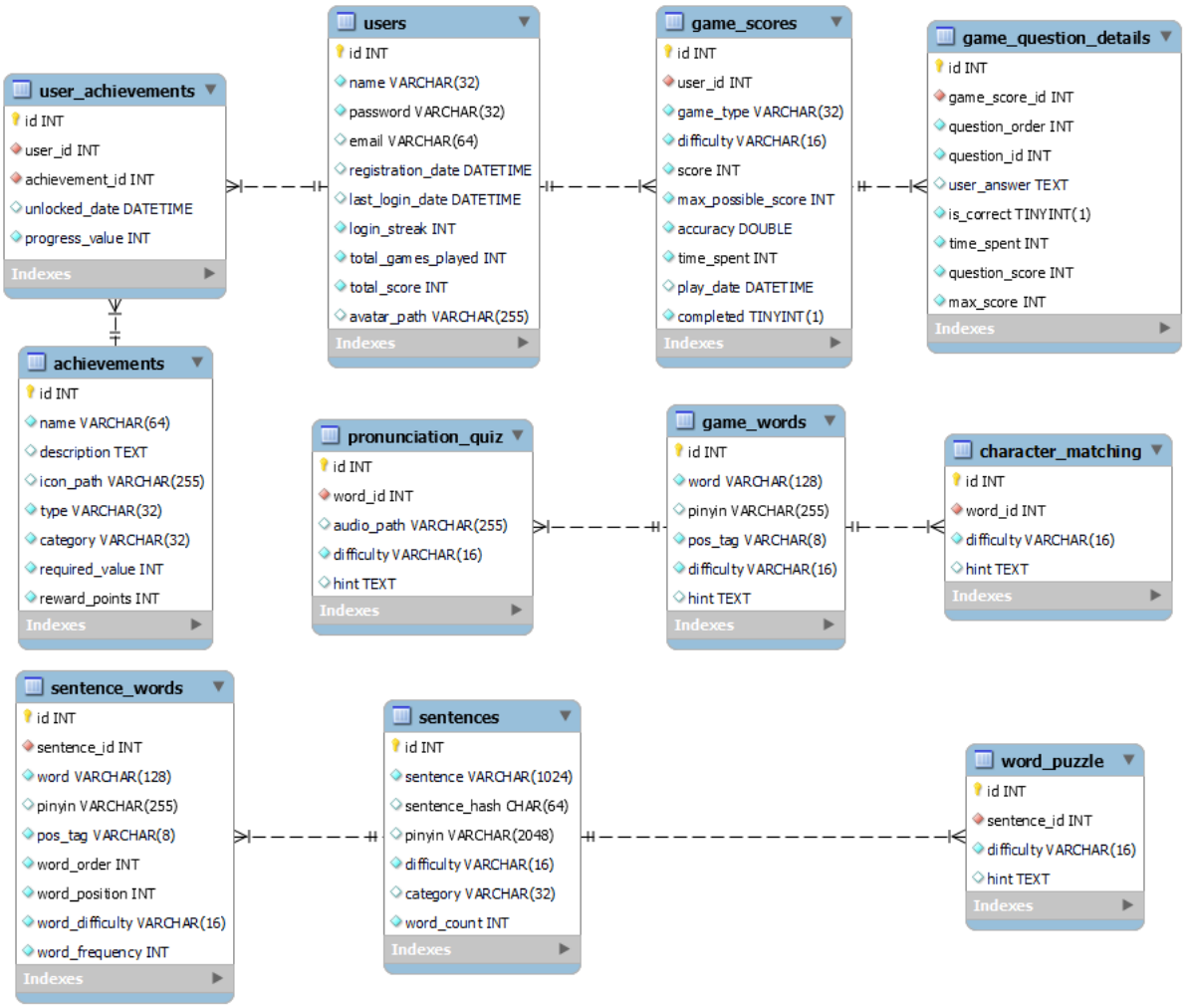


Figure 4.5 ER diagram of the CatLingo database

Table 4.1 Core tables in the CatLingo database

Table name	Main fields	Purpose	Related module
users	id, name, password, email, registration_date, login_streak, total_games_played, total_score	Store learner identity and summary statistics	User management, profile
achievements	id, name, description, type, category, required_value, reward_points	Define achievement rules and rewards	Achievement system
user_achievements	user_id, achievement_id, unlocked_date, progress_value	Link users to unlocked or progressing achievements	Achievement system

game_scores	user_id, game_type, difficulty, score, accuracy, time_spent, play_date, completed	Store one record per game session	All game modules
game_question_details	game_score_id, question_order, question_id, user_answer, is_correct, time_spent, question_score	Store per-question performance	All game modules
sentences	id, sentence, pinyin, difficulty, category, word_count	Store sentence-level resources	Word Puzzle, shared NLP resources
sentence_words	sentence_id, word, pinyin, pos_tag, word_order, word_position, word_difficulty	Store segmented word units	Word Puzzle, distractor logic
character_matching	sentence_id, word_id, difficulty, hint	Store normalized Character Matching references	Character Matching
pronunciation_quiz	sentence_id, word_id, audio_path, difficulty, hint	Store pronunciation question references	Pronunciation Quiz
word_puzzle	sentence_id, difficulty, hint	Store puzzle sentence references	Word Puzzle
game_words	word, pinyin, pos_tag, difficulty, hint	Store direct word prompts for first two games	Character Matching, Pronunciation Quiz

4.4.2 DAO-based Data Access

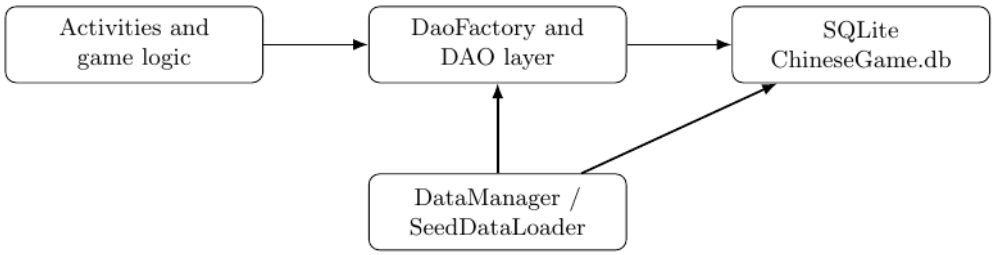
The database layer is not accessed directly from every activity. Instead, CatLingo uses DAOs and DaoFactory to manage persistence-related operations. This design improves maintainability and reduces duplicated SQL logic. It also makes the role of each layer clearer: the activity layer handles interaction, while the DAO layer handles data access.

The repository shows this clearly in several components. DataManager keeps references to UserDao, AchievementDao, GameScoreDao, CharacterMatchingDao, SentenceWordDao, GameWordDao, PronunciationQuizDao, and WordPuzzleDao, and coordinates data reload through them.

This approach is preferable to writing raw SQL in every screen because the application needs to reuse persistent data across multiple modules. For example, game_choice only needs to decide which activity to launch, while GameActivity retrieves session data through DAO-

backed logic rather than embedding all database code in the UI layer.

Figure 4.6 DAO interaction diagram



4.4.3 JSON-to-SQLite Data Import

A key implementation feature of CatLingo is the ability to reload all learning resources from JSON files. This is useful during development because the question bank may change frequently, and manually entering all records into SQLite would be inefficient.

The logic is coordinated through DataManager. First, the class checks whether JSON files in internal storage have changed. If changes are detected, or if a forced reload is requested, the application calls reloadAllDataFromJson(). This method first resets all business tables and SQLite sequences, then imports users, achievements, user-achievements, game scores, and game words. After that, it loads sentence-level resources and finally imports the three game-specific datasets.

```
SeedDataLoader.resetAllTablesAndSequences(context);

userDao.importFromAssetsJson(context);

achievementDao.importAchievementsFromAssetsJson(context);

achievementDao.importUserAchievementsFromAssetsJson(context);

gameScoreDao.importFromAssetsJson(context);

gameWordDao.importFromAssetsJson(context);

SeedDataLoader.loadSentencesAndWordsFromAssets(context);

characterMatchingDao.importFromAssetsJson(context);

pronunciationQuizDao.importFromAssetsJson(context);

wordPuzzleDao.importFromAssetsJson(context);
```

The sentence-loading stage is further refined by SeedDataLoader. This class deletes sentence-dependent tables in foreign-key order, reloads sentences and sentence_words, runs HanLP segmentation for unsegmented records, and recalculates word_position values. This makes the later Word Puzzle logic more reliable and reduces runtime processing during gameplay.

This design is academically valuable because it shows a clear data pipeline rather than only static persistence. It also supports the dissertation structure well: you can describe data preparation as part of system design, and then later evaluate its effectiveness during testing.

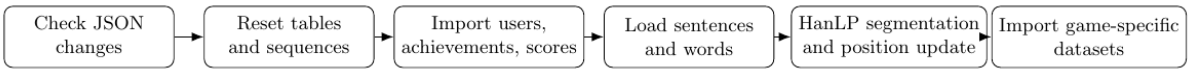


Figure 4.7 JSON-to-SQLite data loading pipeline

4.5 User Management and Game Entry

4.5.1 Login and Registration

The login workflow in CatLingo is implemented in MainActivity. The learner enters a username and password, and the system validates the input before calling userDao.login(username, pwd). If the credentials are correct, login information is updated and the system moves to game_choice with the current USER_ID. If validation fails, a toast message is shown. The same activity also provides entry points for registration, password update, and JSON reload.

This implementation is technically simple, but it is appropriate for the project scope because the app does not require a complex cloud-based account system. In educational terms, it supports persistent learning history while keeping the entry process lightweight.

4.5.2 Difficulty Selection and Game Start

After login, CatLingo enters game_choice, which functions as the central hub of the system. The page presents three buttons for Character Matching, Pronunciation Quiz, and Word Puzzle. Instead of launching the game immediately, the app first displays a difficulty-selection dialog. Once the learner chooses EASY, MEDIUM, or HARD, a short loading dialog appears and the system starts GameActivity with the selected parameters.

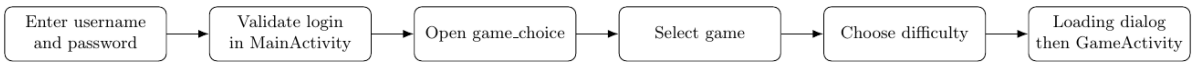


Figure 4.8 User entry and game start flow

Table 4.2 Main activities and responsibilities

Activity / class	Responsibility	Input	Output
MainActivity	Login, register entry, password update entry,	Username, password, button	Transition to game hub or auxiliary

	JSON reload entry	actions	pages
game_choice	Game hub, difficulty selection, loading dialog, menu navigation	Selected game type, selected difficulty	Start GameActivity or open profile/achievement pages
GameActivity	Shared controller for three game modes	GAME_TYPE, USER_ID, DIFFICULTY	Game session flow, feedback, persistence
UserProfileActivity	Show and update user profile plus statistics	USER_ID	Updated profile and returned username if changed
UserAchievementsActivity	Load and render unlocked achievements	USER_ID	Dynamically generated achievement cards
DataManager	Detect changes and reload JSON-backed resources	JSON assets / copied files	Updated SQLite contents
SeedDataLoader	Load sentences, words, segmentation results and positions	sentences.json, sentence_words.json	Structured sentence resources
IseEvaluator	WebSocket-based speech evaluation and XML parsing	Target word, microphone audio	Pronunciation evaluation result

4.6 Character Matching Module

4.6.1 Design

The Character Matching module is designed as an option-based recognition task. For each round, the learner sees a pinyin prompt and selects the corresponding Chinese word from four options. This design strengthens pinyin-to-word mapping and keeps each question short enough for mobile learning.

The core challenge in this module is not the interface itself but the construction of meaningful

distractors. If the wrong options are too different from the correct answer, the task becomes trivial. Therefore, CatLingo selects distractors using the same part-of-speech tag while excluding the correct word. This produces options that are still plausible to the learner.

4.6.2 Implementation

The implementation begins in startCharacterMatchingGame(), which loads ten questions using gameWordDao.getRandomGameWordsForGame(QUESTIONS_PER_GAME, difficulty). A new game_scores record is then created, and placeholder game_question_details rows are inserted for all questions in the session. During each round, showCurrentQuestion() loads the pinyin prompt, retrieves distractor words by POS tag, shuffles the options, and updates the UI. When the learner taps an option, onOptionClick() calculates correctness, updates the detailed record, shows feedback, and moves to the next question.

```
questions = gameWordDao.getRandomGameWordsForGame(QUESTIONS_PER_GAME, difficulty);

currentIndex = 0;

correctCount = 0;

questionResults = new ArrayList<>();

gameCompleted = false;

gameStartTime = System.currentTimeMillis();

GameScore gs = createGameScoreForSession(difficulty, 0, 0, 0.0, 0L, false);

currentGameScoreId = gameScoreDao.save(gs);
```

This is a strong section to include short implementation code because the logic is both clear and educationally meaningful.

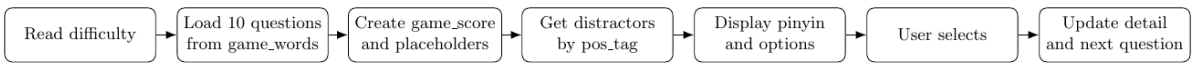


Figure 4.9 Character Matching processing flow

4.7 Pronunciation Quiz Module

4.7.1 Design

The Pronunciation Quiz is designed to give learners immediate speaking feedback while keeping the interaction simple. The learner sees a target word and pinyin, then presses and holds the microphone button to record pronunciation. The app visualises speech intensity through VoiceWaveView, and the result is later converted into a learner-facing score and feedback message. The XML layout confirms this design by defining a dedicated pronunciation panel

containing the target word, pinyin, recognition status, microphone button, score display, and next-question button.

4.7.2 Implementation of the Speech Workflow

The pronunciation workflow begins in initializeSpeechRecognition(), where IseEvaluator is created and connected with the volume visualisation component. During gameplay, startPronunciationQuizGame() loads ten questions and creates a session record in the same way as the Character Matching module. showCurrentPronunciationQuestion() then updates the displayed word, pinyin, progress, and instruction fields.

When the learner presses the microphone button, startRecording() calls iseEvaluator.evaluate(targetWord, callback). The actual evaluation is handled in IseEvaluator, which opens a WebSocket connection, starts audio capture with AudioRecord, sends audio frames to the ISE service, and accumulates the returned XML result. The XML is then parsed to extract total_score, phone_score, fluency_score, integrity_score, and tone_score. Finally, handleIseResult() in GameActivity converts the raw 0–100 score into a 0–10 game score and stores the result in game_question_details.

```
iseEvaluator = new IseEvaluator(
            IFlyTekConfig.APP_ID, IFlyTekConfig.API_KEY, IFlyTekConfig.API_SECRET);
iseEvaluator.setRmsCallback(rmsDb -> {
      if (voiceWaveView != null) voiceWaveView.updateVolume(rmsDb);
});
```

This module is especially suitable for the style of the teacher's examples because it contains a clearly explainable algorithmic flow and a short set of representative implementation steps.

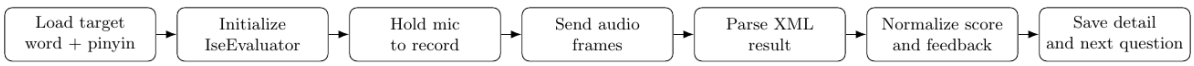


Figure 4.10 Pronunciation Quiz workflow

Table 4.3 Pronunciation evaluation result fields

Field	Source	Meaning	How used in CatLingo
total_score	ISE XML attribute	Overall pronunciation score	Converted to 0-10 game score
phone_score	ISE XML attribute	Segmental or phone-level quality	Included in detailed feedback

fluency_score	ISE XML attribute	Smoothness of reading	Included in detailed feedback
integrity_score	ISE XML attribute	Completeness of spoken response	Included in detailed feedback
tone_score	ISE XML attribute	Tone-related pronunciation quality	Included in detailed feedback
content	ISE XML attribute/string	Recognized reading target	Used in result display and debugging

4.8 Word Puzzle Module

4.8.1 Design

The Word Puzzle module is designed to support sentence-level understanding. Instead of focusing only on vocabulary recognition, it requires learners to reconstruct a correct sentence from shuffled content. This module uses two related strategies. For shorter sentences, the app generates a reorder-style task. For longer sentences or more complex inputs, it switches to a blank-filling style where one content word is removed and replaced with a placeholder. This dual design helps keep the difficulty manageable across different sentence lengths.

The quality of this module depends heavily on sentence preprocessing. The app retrieves token-level information from sentence_words, which is generated during JSON import and HanLP segmentation. If token data are unavailable, the system falls back to character-based splitting. This fallback design increases robustness.

4.8.2 Implementation

The module begins with loadWordPuzzleQuestions(), which selects sentences from SQLite according to the chosen difficulty. Each sentence is passed to buildWordPuzzleQuestion(). That method first trims the sentence, retrieves token-level data through getSentenceTokens(), and then decides whether to generate a reorder question or a blank question. buildShortSentenceReorderQuestion() creates distractor options by shuffling tokens and retrieving other sentence distractors if necessary, while buildLongSentenceBlankQuestion() masks one content word and creates candidate options around it.

```
List<SentenceToken> tokens = getSentenceTokens(sentenceId, sentenceText);

boolean useBlankMode = sentenceText.length() >= LONG_SENTENCE_CHAR_THRESHOLD

        || tokens.size() >= LONG_SENTENCE_WORD_THRESHOLD;
```

```
if (useBlankMode) {
    return buildLongSentenceBlankQuestion(sentenceId, sentenceText, difficulty, tokens);
}
return buildShortSentenceReorderQuestion(sentenceId, sentenceText, difficulty, tokens);
```

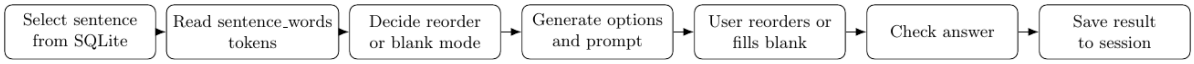


Figure 4.11 Word Puzzle generation pipeline

This is an especially strong place to add code in the dissertation because the control logic clearly reflects the design decisions described in the text.

Table 4.4 Word Puzzle modes in CatLingo

Mode	Trigger condition	User task	Example output format
Reorder mode	Shorter sentence or fewer tokens	Rearrange blocks into correct word order	Five shuffled blocks to drag into slots
Blank mode	Longer sentence or many tokens	Choose a missing content word from options	Sentence prompt with one placeholder and option set

4.9Achievement and Progress Tracking

4.9.1 Design

The achievement and progress subsystem is intended to transform separate play sessions into a continuous learning record. The design includes persistent game-score storage, per-question detail storage, user statistics, and achievement rendering. Instead of treating achievements as a purely decorative feature, CatLingo links them to stored session data and makes them visible through a dedicated activity.

4.9.2 Implementation

The implementation is distributed across several parts of the system. Session-level records are stored in game_scores, and question-level records are stored in game_question_details, as shown in the Character Matching and Pronunciation Quiz logic. User profile statistics are loaded in UserProfileActivity, where the app shows username, email, registration date, total games played, total score, and login streak. The activity also supports editing the username and email with validation logic.

```
List<UserAchievement> userAchievements = achievementDao.getUserAchievements(userId);
for (UserAchievement userAchievement : userAchievements) {
    Achievement achievement = achievementDao.findById(userAchievement.getAchievementId());
    if (achievement != null) {
        View achievementCard = createAchievementCard(achievement, userAchievement);
        achievementsContainer.addView(achievementCard);
    }
}
```

Achievement display is implemented in UserAchievementsActivity. The method loadUserAchievements() retrieves all unlocked achievements for the current user. For each achievement, createAchievementCard(...) dynamically constructs a card containing the name, description, category, type, unlock time, and reward points. This is an elegant implementation because it turns persistent database data into a structured visual component rather than relying on hard-coded static pages.

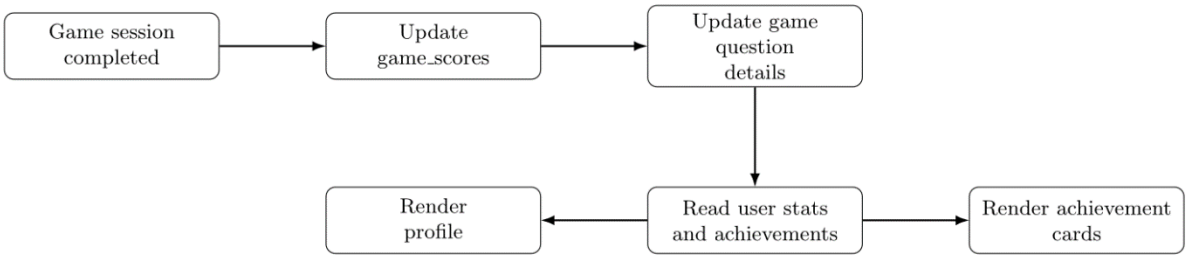


Figure 4.12 Achievement and progress data flow

Table 4.5 Progress and achievement data sources

UI element	Data source	Update trigger	User-facing purpose
Total games played	users.total_games_played	After session update and user statistics refresh	Show long-term usage
Total score	users.total_score / game_scores	After session completion	Show cumulative performance
Login streak	users.login_streak	On successful login	Encourage daily use
Achievement list	user_achievements + achievements	When achievement page is opened	Show unlocked rewards
Per-session result	game_scores	At end of each game session	Show session outcome

Per-question detail	game_question_details	After each question	Support detailed performance tracking

4.10 UI and User Experience Considerations

Although screenshots should be reserved for the Results and Discussion chapter, the design and implementation chapter should still explain the UI logic of CatLingo. The repository shows that the app uses a consistent visual language across the login page, game hub, and game panels. activity_main.xml uses a mascot icon, a title, a subtitle, a card-style login form, and a vertical action panel. activity_game_choice.xml continues the same visual style with a toolbar, mascot, subtitle, and three large game buttons. The shared activity_game.xml layout also follows a panel-based design, where only the relevant game panel is shown according to the selected mode.

This consistency matters because CatLingo is not only a technical prototype but also a learner-facing mobile application. The use of a single visual language reduces cognitive load and supports smooth switching between modules.

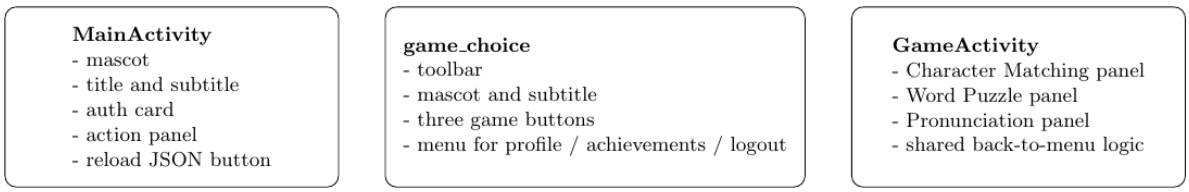


Figure 4.13 UI component organisation of CatLingo

4.11 Chapter Summary

This chapter has presented the design and implementation of CatLingo in a form that is more closely aligned with the project guidance and the style of the supplied examples. It explained the layered system architecture, the activity-based navigation model, the DAO-backed persistence design, and the JSON-to-SQLite content-loading pipeline. It then described the implementation of the main user-management flow, the three core game modules, and the achievement plus progress subsystem.

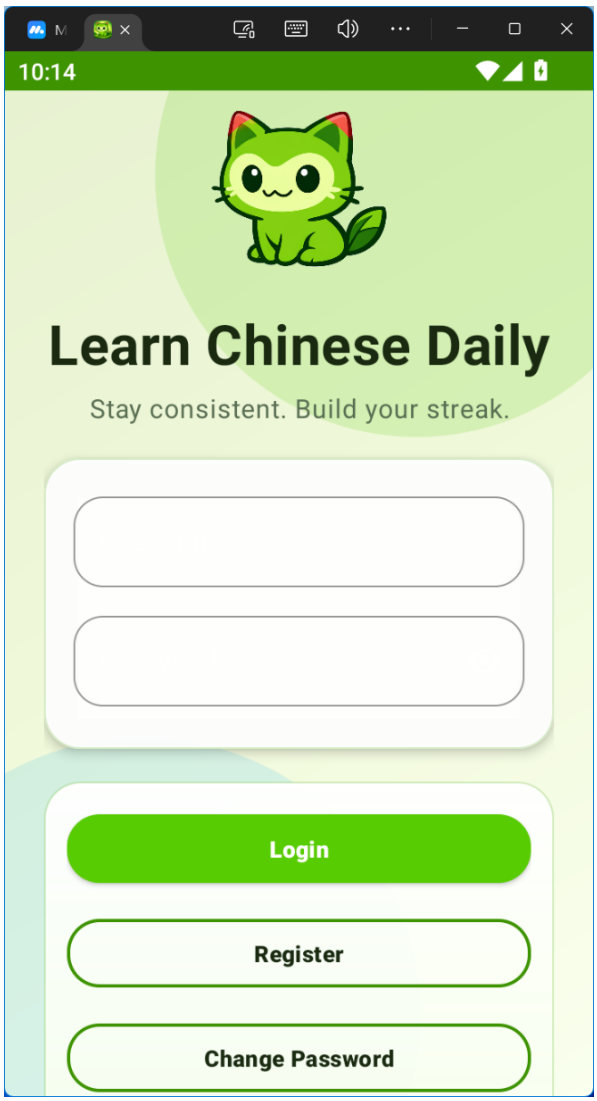
Most importantly, the chapter demonstrated that CatLingo is not simply a collection of screens. Instead, it is a structured Android educational system in which interface logic, gameplay logic, persistence, and external speech evaluation are coordinated through a modular design. The next chapter can therefore evaluate the effectiveness of this implementation through testing and results analysis.

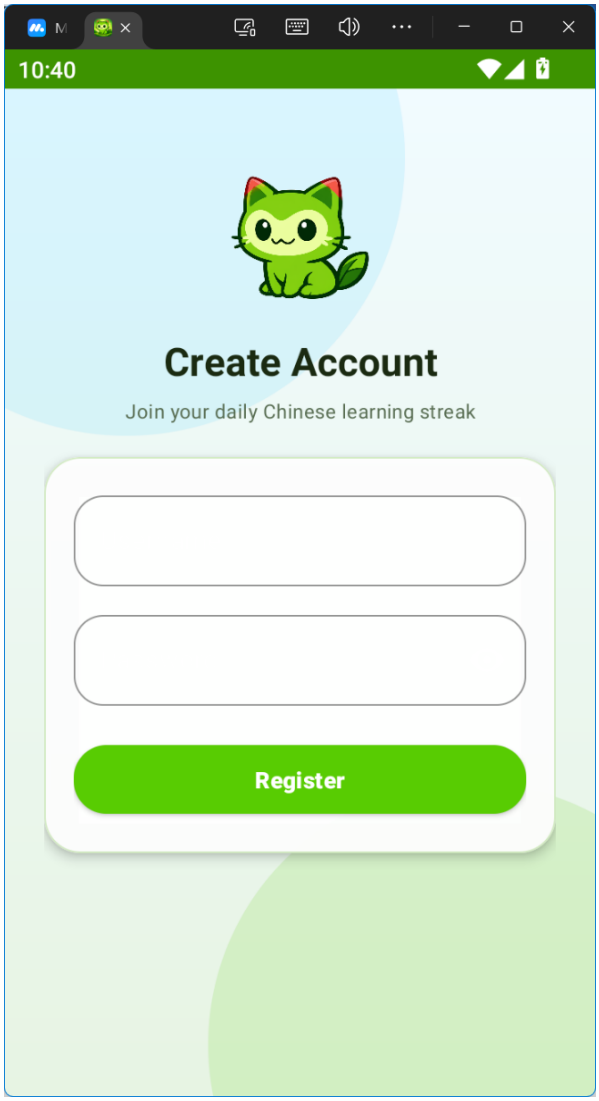
Chapter 5 Results and Discussion

5.1 User Management Results

The implemented user-management subsystem demonstrates that CatLingo is able to function as a persistent learning application rather than as a one-time demo. The login page supports credential validation, transitions to the game hub, and auxiliary actions such as registration entry, password update entry, and JSON reload. The profile page further extends this by displaying registration time, total games played, total score, and login streak, as well as allowing the learner to update username and email subject to validation checks.

From a design perspective, this result is important because it means that later learning data are attached to a stable user identity. This supports educational continuity and makes progress tracking meaningful. Compared with a purely anonymous mini-game collection, the current system therefore offers a stronger foundation for learner retention.



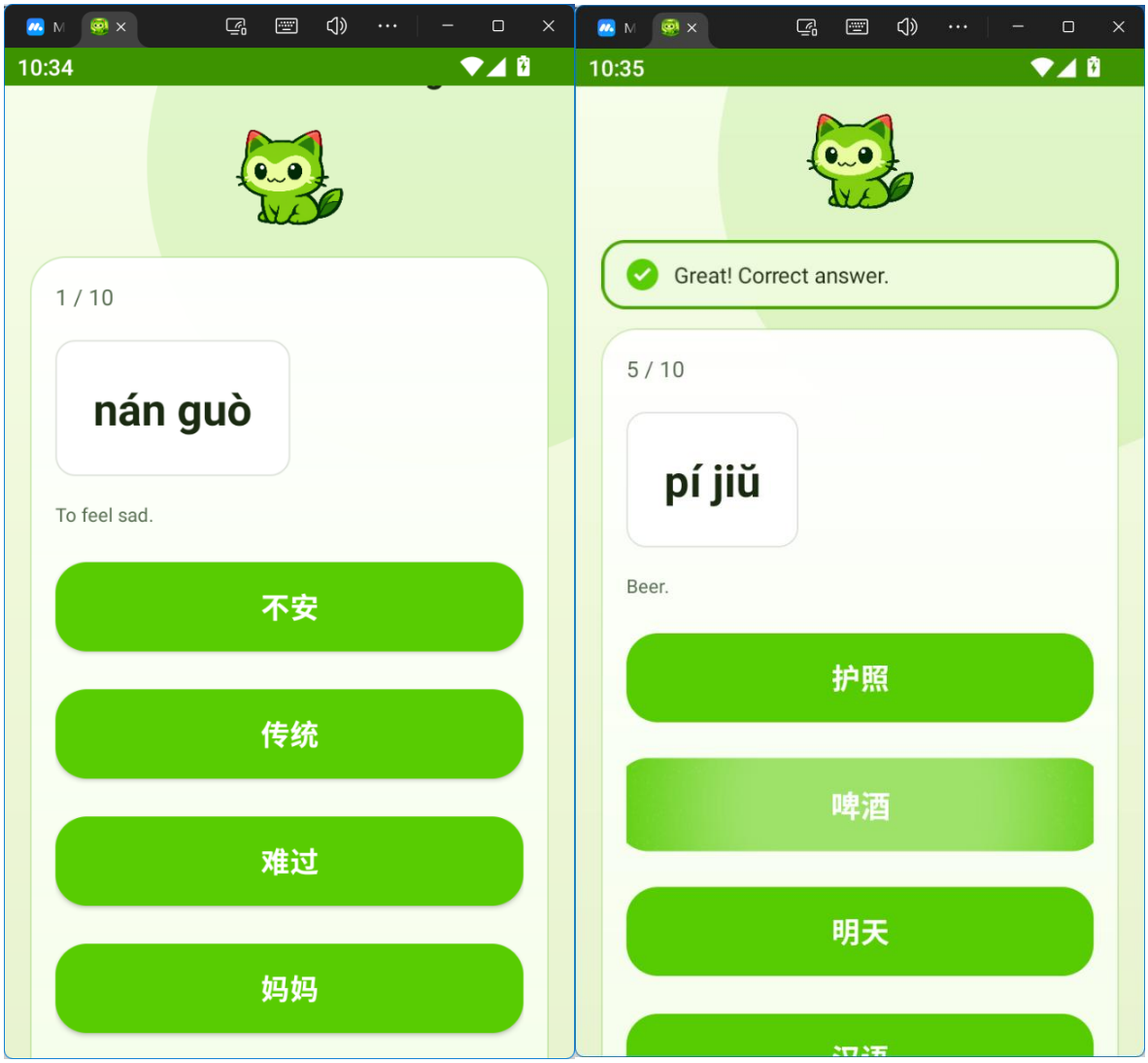


5.2 Character Matching Results

The Character Matching module produced the clearest example of a complete learning loop. The app can load a session, generate plausible distractors, update feedback after each answer, and save the session result. The system also records incomplete sessions if the learner leaves early, which reflects a practical understanding of mobile learning behaviour.

As a learning activity, this module appears effective for early-stage vocabulary reinforcement because it connects pinyin with written form in a direct and low-friction way. It is also the module with the simplest and most stable runtime behaviour, since it depends entirely on local resources and does not require third-party services. For this reason, Character Matching is arguably the most mature component in the current prototype.

A limitation is that the quality of the exercise depends on the quality and balance of the underlying lexical dataset. If the POS-tag distribution is uneven, distractors may sometimes be less effective than intended.

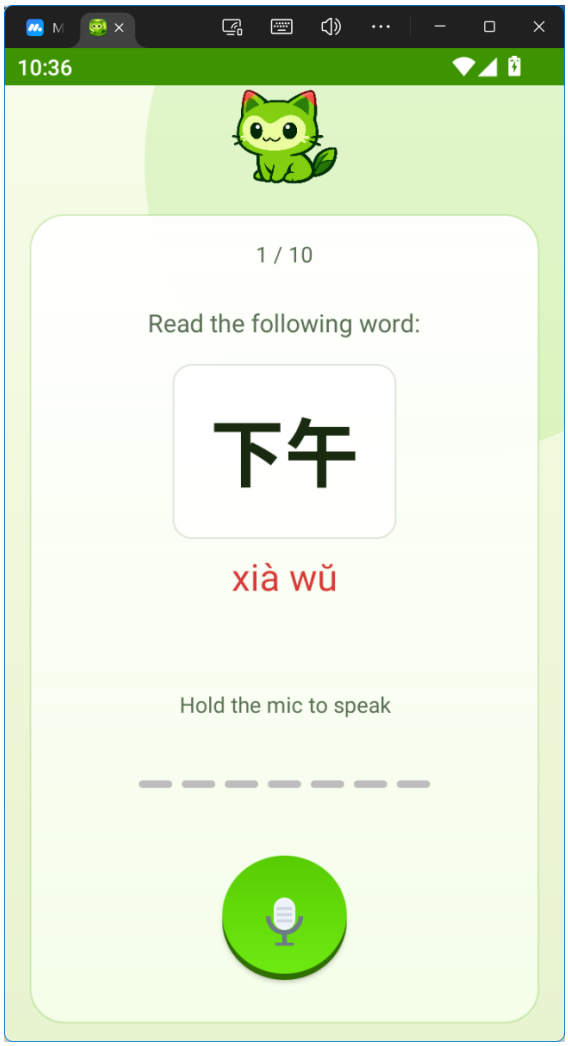


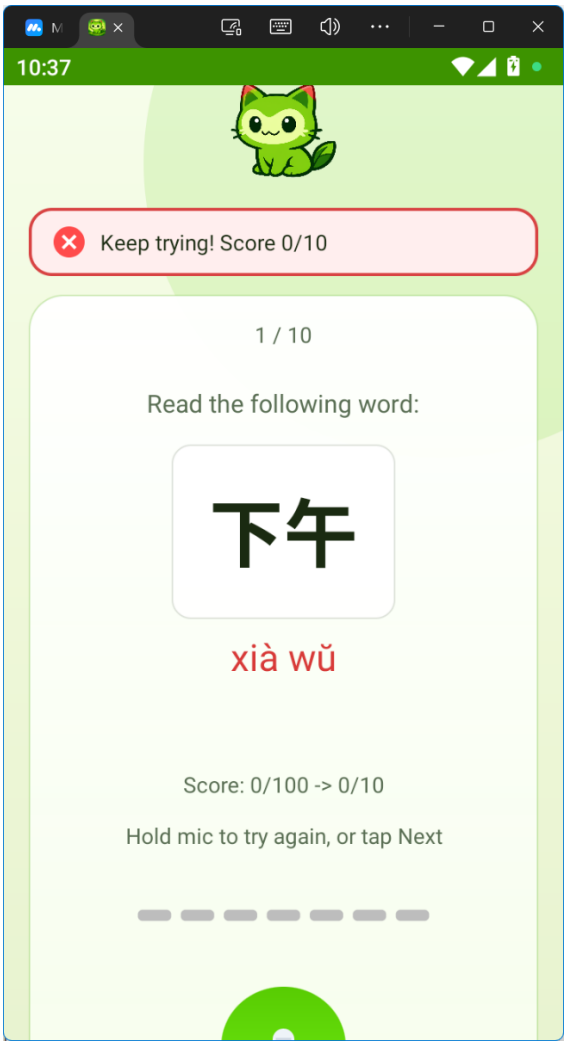
5.3 Pronunciation Quiz Results

The Pronunciation Quiz demonstrates one of the most technically ambitious parts of the project. It is able to display a target item, capture speech through the microphone, evaluate the recording through a WebSocket-based ISE workflow, and convert the returned XML result into learner-visible feedback and an internal score.

This result is significant because it moves the application beyond text-only interaction. The learner is not merely shown a word but is invited to perform spoken output, which is then processed and reflected in the system's scoring logic. The use of both a 0–100 raw score and a normalised 0–10 game score helps connect pronunciation practice with the wider gamification model of CatLingo.

However, this module also reveals the main technical limitation of the prototype. Pronunciation results can vary depending on speaking environment, connection quality, and service behaviour. Therefore, while the module is functionally impressive and meaningful within the scope of the project, its evaluation output should still be treated as supportive rather than absolute.



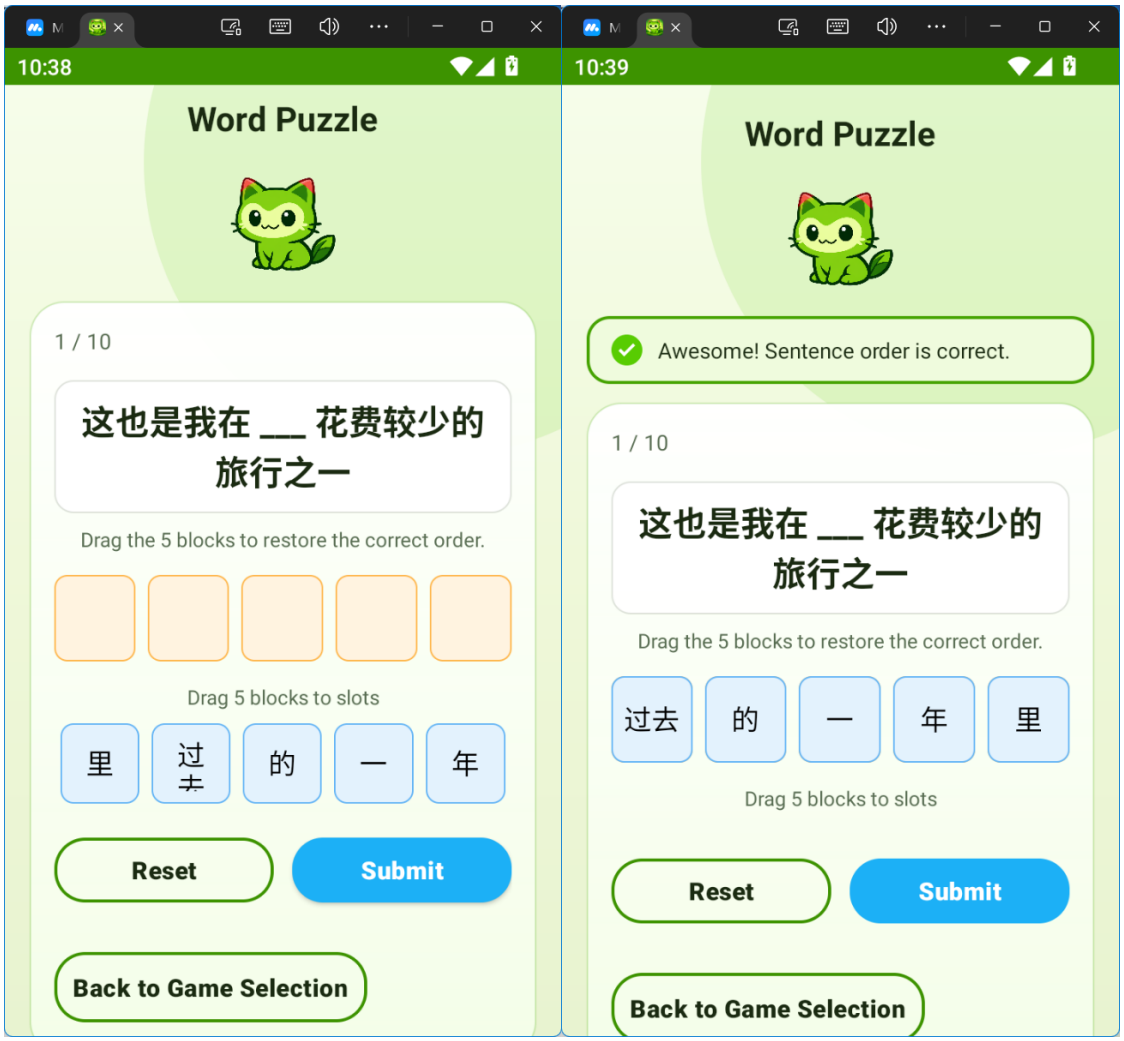


5.4 Word Puzzle Results

The Word Puzzle module demonstrates that CatLingo can support sentence-level learning rather than focusing only on isolated vocabulary. By retrieving sentence resources from SQLite, reading token-level information from sentence_words, and switching between reorder mode and blank mode based on sentence complexity, the module turns stored language resources into interactive exercises.

This is an important result because sentence reconstruction encourages a deeper level of processing than simple recognition. It helps learners think about sequence, lexical grouping, and sentence structure. The token fallback strategy also increases robustness: if token data are unavailable, the app can still generate a usable, though less linguistically refined, question.

The current limitation is that learning quality depends heavily on preprocessing quality. If token boundaries or distractors are not pedagogically natural, the task may become confusing even if it is technically correct. This means the module has strong potential but would benefit from further content tuning.

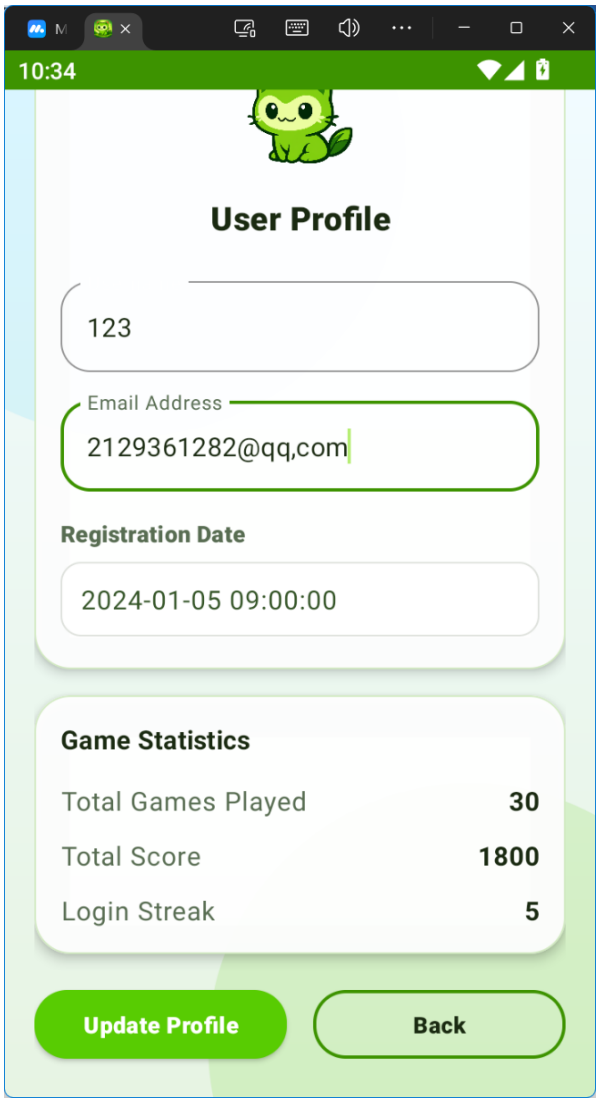


5.5 Achievement and Progress Tracking Results

The results of the achievement and progress subsystem show that CatLingo is able to connect repeated use with visible learner records. The profile page presents accumulated statistics, while the achievement page dynamically generates cards for unlocked achievements, including name, description, category, type, unlock time, and reward points.

This design is effective because it gives the learner a sense that the app remembers prior effort. Even without advanced adaptive learning, this persistence contributes to motivation. It also supports one of the project's central objectives: using gamification not as a superficial interface feature, but as a mechanism for sustained engagement.

The main limitation is that the current dissertation evidence is strongest for achievement display rather than for a large and varied set of automatically triggered reward scenarios. Further development should therefore strengthen rule coverage and test more diverse user histories.



5.6 Discussion of System Effectiveness

Taken together, the results suggest that CatLingo is effective as a modular educational prototype for beginner and intermediate Chinese learners. The app successfully combines three distinct learning tasks in one Android system and supports them with user management, structured data persistence, and visible progress representation. From a software-engineering perspective, this is already a meaningful achievement, because many student projects remain at the level of isolated demonstrations rather than integrated applications.

From a pedagogical perspective, the system also shows useful diversity. Character Matching supports recognition, Pronunciation Quiz supports speaking practice, and Word Puzzle supports sentence-level understanding. This multi-function design responds well to the original project motivation, which aimed to address the fragmentation of many existing Chinese learning tools.

At the same time, the effectiveness of the system should not be overstated. The project currently demonstrates feasibility and internal coherence more strongly than large-scale learning effectiveness. There is no evidence yet of long-term user-study outcomes, and the pronunciation module in particular requires cautious interpretation.

5.7 Discussion of Limitations

Several limitations remain visible in the current version of CatLingo. First, the project still relies on local persistence only. This keeps the architecture simple and appropriate for the final-year project, but it limits broader deployment scenarios. Second, the speaking module depends on external evaluation behaviour, so the system cannot fully control or explain every pronunciation result. Third, question quality still depends on the imported dataset and preprocessing pipeline. This is especially important for distractor generation and sentence-based tasks.

There is also a presentation-level limitation: while the UI has become more consistent, some final polish and cross-device refinement are still needed. These limitations do not invalidate the project, but they define the boundary between a successful prototype and a finished product.

5.8 Comparison with Initial Objectives

The initial objectives of the project were to analyse learner needs, build a modular Android architecture, implement three core mini-games, integrate a local database and structured question management, and support repeated learner engagement through progress and reward features. Based on the current implementation, these objectives have been substantially achieved.

The architecture objective has been met through the layered Android design and the single-container GameActivity. The persistence objective has been met through SQLite, DAO-backed access, and JSON-driven import. The three educational mini-games are implemented in a usable form, and progress-related screens are present.

The only objective that remains only partially achieved is full evaluation at scale. The system can be tested and discussed, but its long-term educational effectiveness still requires broader user-based evidence.

5.9 Summary

This chapter discussed the practical results of CatLingo and interpreted them in relation to the project goals. The results show that the system already supports a meaningful educational workflow across user management, vocabulary recognition, pronunciation practice, sentence reconstruction, and visible progress. The strongest aspects of the current prototype are its integrated structure, local persistence, and technically grounded speaking workflow. The main limitations concern scale, polish, and the constraints of a prototype evaluation setting. These findings lead naturally to the final conclusion of the dissertation.

Chapter 6 Conclusion and Future Work

6.1 Overall Project Summary

This dissertation presented the design and implementation of **CatLingo**, a gamified Android application intended to support beginner and intermediate learners of Chinese. The project was motivated by the observation that many existing Chinese learning applications remain fragmented, offering isolated functions but limited integration of recognition, pronunciation, sentence-level practice, and progress tracking. In response, the project developed an Android system that combines user management, three mini-games, local persistence, and achievement-based motivation within a single application.

The completed implementation demonstrates that these features can be integrated coherently in a native Android environment. The system includes login and profile management, difficulty-based game selection, Character Matching, Pronunciation Quiz, Word Puzzle, and visible achievement and progress pages. It also includes a meaningful data pipeline from JSON resources into SQLite, together with speech evaluation and sentence preprocessing support.

6.2 Achievement of Objectives

The project's main objectives have largely been achieved. First, the project identified core learner needs in character recognition, pronunciation, and sentence understanding, and translated them into a practical set of modules. Second, it established a modular Android architecture that separates UI logic, gameplay control, persistence, and external service integration. Third, it implemented three distinct learning activities rather than only one. Fourth, it created a local data model capable of storing not only users and scores, but also detailed question-level performance and structured sentence resources. Finally, it integrated gamification through score persistence, profile statistics, and achievement display.

The project therefore succeeds both as a software implementation and as a proof of concept for a Chinese learning application with multiple interactive components.

6.3 Main Limitations

Despite these achievements, several limitations remain. The first is evaluation scope: the current dissertation demonstrates technical feasibility and internal correctness more strongly than long-term pedagogical impact. A second limitation is the dependency of the pronunciation module on hardware conditions and external service behaviour. A third limitation is that the system currently stores data locally only, which fits the project scope but limits broader deployment.

Finally, content quality remains partly dependent on the imported datasets and preprocessing results, especially in modules that rely on distractors or segmented sentence units.

These limitations are typical of a final-year project prototype and should be interpreted as opportunities for further refinement rather than as fundamental design failures.

6.4 Future Work

Several directions for future work are clear. First, the system could be extended with a cloud-backed account and synchronisation mechanism, allowing learners to continue across devices. Second, the achievement subsystem could be expanded with more detailed unlock logic and better integration with cumulative user behaviour. Third, the Word Puzzle module could be improved through stronger content curation, more advanced difficulty modelling, and richer interaction styles. Fourth, the pronunciation module could be enhanced by adding more detailed learner feedback and testing it under a wider range of device and environment conditions. Finally, a larger user study could be conducted to evaluate the learning impact and usability of CatLingo more formally.

In this sense, the present project should be seen as a strong foundation rather than a finished endpoint. It has already shown that a gamified Chinese learning application can be built in a modular and technically meaningful way on Android, and it provides a clear platform for future expansion.

References

Acknowledgement

Appendices

Disclaimer

This report is submitted as part requirement for the undergraduate degree programme at Queen Mary University of London, and Beijing University of Posts and Telecommunications. It is the product of my own labour except where indicated in the text. The report may be freely copied and distributed provided the source is acknowledged.

BUPT No.:
QMUL No.:
Full Name (Pin Yin):
Full Name (Chinese):

Signature:

Date: DD-MM-YYYY

Acknowledgement of using Generative AI

You must acknowledge where you used AI in the final report, including where you have directly quoted or paraphrased AI-generated content or used tools to summarise readings, brainstorm ideas, plan your process, or for editing or proofreading purposes.

Write one acknowledgement for each GenAI system you used. Summarise the acknowledge of all AI systems that you used into a table, structured as follows:

Publisher	**Name, version, URL**	**Use**	**Section**
Company that made the AI system	Name, version, and URL of the generative AI system	Brief description of context in which the tool was used. (one paragraph)	Where it is used (section number or page number)
…	…	…	…

Here is an example:

I acknowledge the use of the following Generative AI tools

Publisher	**Name, version, URL**	**Use**	**Section**
DeepSeek	DeepSeek V3.2 https://www.deepseek.com/	To search materials for background study in the drafting of this report	Chapter 2, Section 2.1, Page 10
iFLYTEK	iFLYTEK Spark X1 https://xinghuo.xfyun.cn/	To summarise my initial notes and to proofread my final draft.	Chapter 5, Section 5.1, Page 34-35

I confirm the submitted work is my own work.

Project specification

Include your project specification, part 1 and part 2 here. It must be the final version submitted to QMPlus.

Early-term progress report

Include your project early-term progress report here. It must be the final version submitted to QMPlus.

Mid-term progress report

Include your project mid-term progress report here. It must be the final version submitted to QMPlus.

Supervision log

Include your project supervision log here.

Additional appendices (as needed)

Information that you think may be helpful or relevant for the reader but that is not directly relevant to the story of your project. Things that might be suitable as an appendix to a report are:

- Large tables of numerical results that have been displayed graphically in the main body of the report.
- Important parts of datasheets for specific devices you have used in your project if you think that they are important enough that the reader should have access to them without finding them off the web themselves.
- Mathematical proofs and results that are important to show but not important to the flow of the story in the report.

NOTE: Full code listings must NOT be included as an appendix, but extracts of code may be included in the body of the report to illustrate particular points. Code should be submitted as supporting documents to QMPlus.

Risk and Environmental Impact Assessment

Please refer to the project handbook section 3.6.12.